

Arsitektur Proyek Go: Mengelola Module, Package, Access Modifier, dan Standar Struktur Folder

Durasi: 3 Jam (180 Menit) **Level:** Intermediate (Menengah)

Ketika aplikasi yang Anda bangun berkembang dari skrip satu berkas (*single file*) menjadi layanan backend skala produksi dengan ribuan baris kode, cara Anda mengorganisasikan berkas-berkas tersebut menjadi penentu utama kualitas proyek. Go memiliki aturan yang sangat teratur dalam mengelompokkan kode.

Modul ini akan membedah konsep **Go Modules** untuk manajemen pustaka pihak ketiga, **Packages** untuk modularisasi kode, **Access Modifiers** untuk menyembunyikan detail implementasi data, serta **Standard Project Layout** (Struktur Folder Standar) yang diakui oleh komunitas Go global untuk membangun backend yang profesional, bersih, dan mudah dirawat.

Sesi 1: Go Modules & Go Packages Deep Dive (45 Menit)

Go mengelola modul dan pustaka menggunakan dua konsep utama: **Module** (sebagai unit distribusi dan penentu versi) dan **Package** (sebagai unit kompilasi lokal).

1. Go Modules (`go.mod` dan `go.sum`)

Sebuah modul Go ditandai dengan keberadaan berkas `go.mod` di direktori akar (*root*) proyek Anda.

- `go.mod` : Menyimpan nama modul utama (identitas proyek) beserta daftar dependensi pustaka luar dan versinya.
- `go.sum` : Menyimpan tanda hash (*checksum*) dari kode pustaka luar untuk memastikan integritas dan keamanan pustaka saat diunduh ulang.

Perintah CLI Go Modules yang sering digunakan:

```
go mod init <nama_modul>    # Membuat modul baru
go mod tidy                  # Mengunduh dependensi baru & menghapus yang tidak digunakan
go get <url_pustaka>         # Mengunduh pustaka spesifik (misal: github.com/gin-gonic/gin)
```

2. Go Packages

Package adalah kumpulan berkas kode Go (berakhiran `.go`) yang berada di dalam **satu direktori yang sama** dan memiliki baris deklarasi package yang identik di baris teratas berkasnya.

Aturan Penting Package Go:

1. Semua berkas di dalam satu direktori fisik **wajib** memiliki nama package yang sama.
2. Nama package biasanya disamakan dengan nama direktori fisiknya untuk memudahkan pemanggilan.
3. Untuk mengimpor package lokal, gunakan path absolut modul utama diikuti path direktori package-nya:

```
import "nama_modul_utama/path/ke/nama_package"
```

3. Fungsi `init()`

Fungsi `init()` adalah fungsi khusus di Go yang otomatis dieksekusi **satu kali** saat package pertama kali di-load ke memori, bahkan sebelum fungsi `main()` dijalankan.

```
package config

import "fmt"

func init() {
    fmt.Println("Konfigurasi database diinisialisasi otomatis...")
}
```

Sesi 2: Access Modifier di Go (45 Menit)

Tidak seperti bahasa berorientasi objek lain yang memiliki kata kunci `public`, `private`, atau `protected`, Go menggunakan aturan yang sangat sederhana dan elegan: **kapitalisasi huruf pertama** dari pengenal (*identifier*) tersebut.

1. Exported Identifiers (Public)

Jika nama fungsi, struct, interface, field struct, variabel, atau konstanta dimulai dengan **huruf kapital (A-Z)**, maka pengenal tersebut **diekspor** (*exported*). Artinya, pengenal tersebut dapat diakses dan digunakan oleh package lain.

```
package db

// DatabaseConnection bertipe Exported karena huruf pertama kapital
type DatabaseConnection struct {
    URL string // Exported Field
}

// Connect bertipe Exported
func Connect() {
    // ...
}
```

2. Unexported Identifiers (Private)

Jika namanya dimulai dengan **huruf kecil (a-z)**, maka pengenal tersebut **tidak diekspor** (*unexported*). Artinya, ia hanya dapat diakses di dalam package yang sama. Package luar tidak diizinkan membaca atau memodifikasinya.

```
package db

// connectionString tidak bisa diakses dari luar package db
var connectionString = "postgres:// ..."

// validateConnection hanya untuk internal package db
func validateConnection() bool {
    return true
}
```

Sesi 3: Standar Struktur Folder Proyek Go (45 Menit)

Komunitas Go secara global menyepakati sebuah pola tata letak folder yang rapi (*Standard Go Project Layout*) untuk mengelola proyek berskala menengah hingga besar. Struktur ini memisahkan kode eksekusi, kode publik, dan kode internal aplikasi.

Berikut adalah struktur direktori standar yang disederhanakan untuk backend e-commerce:

```
/my-shop-api
├── go.mod           # File modul utama
├── cmd/             # Entry point aplikasi utama
│   └── api/
│       └── main.go  # Berkas utama untuk menjalankan aplikasi web/API
├── internal/        # Kode internal aplikasi (tidak boleh di-import oleh proyek)
│   ├── model/       # Definisi model data/struct (e.g. Product, Customer)
│   ├── service/     # Logika bisnis inti aplikasi (e.g. Cart, Checkout)
│   └── auth/         # Sistem otorisasi & verifikasi token
└── configs/         # File konfigurasi sistem (.env, .yaml)
```

Sesi 4: Praktikum Mandiri: "Membangun Proyek E-Commerce Terstruktur" (45 Menit)

Mari kita praktikkan pembuatan proyek modular terstruktur dengan membuat struktur direktori di atas dan menyusun kodenya secara bertahap.

Langkah 1: Struktur File Proyek

Buatlah folder proyek baru dan buat struktur file berikut:

```
/go-shop (Direktori Utama)
├── go.mod
├── internal
│   ├── model
│   │   └── product.go
│   └── service
│       └── cart.go
└── cmd
    └── api
        └── main.go
```

Langkah 2: Menulis Modul Utama (`go.mod`)

Jalankan inialisasi modul di terminal root `/go-shop` :

```
go mod init go-shop
```

(Atau buat file `go.mod` secara manual dengan isi modul `go-shop`)

Langkah 3: Menulis Model Produk (`internal/model/product.go`)

Buat file `product.go` untuk mendefinisikan model produk e-commerce. Kita menggunakan field-field kapital agar bisa dibaca dari package lain.

```
package model

import "errors"

// Product bertipe Exported
type Product struct {
    SKU    string
    Name   string
    Price  float64
    Stock  int
}

// Method untuk validasi data (Exported)
func (p *Product) Validate() error {
    if p.SKU == "" || p.Name == "" {
        return errors.New("invalid product: SKU dan nama produk tidak boleh kosong")
    }
    if p.Price < 0 || p.Stock < 0 {
        return errors.New("invalid product: harga dan stok tidak boleh bernilai negatif")
    }
    return nil
}
```

Langkah 4: Menulis Layanan Keranjang (`internal/service/cart.go`)

Buat file `cart.go` untuk logika bisnis e-commerce. Perhatikan cara kita mengimpor tipe `Product` dari package `model`.

```

package service

import (
    "errors"
    "fmt"
    "go-shop/internal/model" // Import package lokal model
)

type CartService struct {
    Items map[string]int // Map: SKU → Kuantitas
}

// Constructor CartService
func NewCartService() *CartService {
    return &CartService{
        Items: make(map[string]int),
    }
}

// AddItem bertipe Exported untuk diakses dari cmd/api/main.go
func (c *CartService) AddItem(prod model.Product, qty int) error {
    // Panggil validasi model
    if err := prod.Validate(); err != nil {
        return err
    }
    if qty ≤ 0 {
        return errors.New("kuantitas belanja harus lebih dari nol")
    }
    if prod.Stock < qty {
        return fmt.Errorf("stok produk '%s' tidak cukup (Sisa: %d)", prod.Name,
    }

    // Tambahkan kuantitas ke map keranjang
    c.Items[prod.SKU] += qty
    return nil
}

```

Langkah 5: Menulis Entry Point Utama (cmd/api/main.go)

Rakit semua komponen di file `main.go` . Di sini kita mengimpor kedua package lokal yang telah dibuat.

```

package main

import (
    "fmt"
    "go-shop/internal/model"    // Mengimpor model
    "go-shop/internal/service" // Mengimpor service
)

func main() {
    fmt.Println("=====")
    fmt.Println("        APLIKASI KASIR GO-SHOP (MODULAR LAYOUT)        ")
    fmt.Println("=====")

    // 1. Definisikan Produk
    laptop := model.Product{
        SKU:    "SKU-LAP-01",
        Name:   "Laptop Business Pro",
        Price:  12500000.0,
        Stock:  5,
    }

    mouse := model.Product{
        SKU:    "SKU-MOU-02",
        Name:   "Mouse Wireless Silent",
        Price:  250000.0,
        Stock:  20,
    }

    // 2. Inisialisasi Service Keranjang Belanja
    cart := service.NewCartService()

    // 3. Masukkan Barang ke Keranjang
    fmt.Println("[PROSES] Memasukkan Laptop x2 ke keranjang...")
    if err := cart.AddItem(laptop, 2); err != nil {
        fmt.Println("Gagal:", err)
    }

    fmt.Println("[PROSES] Memasukkan Mouse x5 ke keranjang...")
    if err := cart.AddItem(mouse, 5); err != nil {
        fmt.Println("Gagal:", err)
    }

    // 4. Simulasi Pembelian Melebihi Stok
    fmt.Println("[PROSES] Memasukkan Laptop x10 ke keranjang...")
    if err := cart.AddItem(laptop, 10); err != nil {
        fmt.Println("Gagal:", err) // Diharapkan mencetak stok tidak cukup
    }

    // 5. Tampilkan Keranjang Belanja Akhir
    fmt.Println("\nRincian Akhir Keranjang Belanja:")
    for sku, qty := range cart.Items {
        fmt.Printf("- SKU: %-12s | Kuantitas: %2d unit\n", sku, qty)
    }
}

```

```
}  
    fmt.Println("=====")  
}
```

Cara Menjalankan Proyek:

Buka terminal di root `/go-shop` dan jalankan:

```
go run cmd/api/main.go
```

Sesi 5: Soal Latihan Praktik E-Commerce (Mandiri)

Selesaikan dua latihan pemrograman e-commerce tingkat lanjut di bawah ini untuk menguji pemahaman Anda mengenai integrasi Package, Access Modifier, dan struktur folder.

Latihan 1: Pembuatan Package Otorisasi Merchant (`internal/auth`)

Deskripsi Soal: Buatlah sebuah sistem otorisasi token sederhana untuk merchant yang dikelompokkan ke dalam package tersendiri untuk menjaga keamanan data token.

Kriteria Implementasi:

1. Buatlah direktori baru `/internal/auth` dan buat berkas `auth.go` di dalamnya.
2. Di dalam berkas `auth.go`, buatlah interface `Authenticator` (Exported) yang mendefinisikan method:
 - `ValidateToken(token string) bool`
3. Buat struct konkret `MerchantAuth` yang memiliki field `tokenKey string` (Unexported/Private agar tidak bisa diubah langsung dari luar package).
4. Buat fungsi konstruktor `NewAuthenticator(secretKey string) Authenticator` (Exported) untuk menginstansiasi objek pointer `MerchantAuth` dengan token rahasia tertentu.
5. Implementasikan method `ValidateToken(token string) bool` pada struct `MerchantAuth` untuk memeriksa apakah parameter `token` yang dikirim cocok dengan nilai `tokenKey`.
6. Import package `go-shop/internal/auth` di dalam berkas main `cmd/api/main.go`. Lakukan pengujian otorisasi di dalam `main()` menggunakan token valid dan tidak valid sebelum memproses transaksi belanja kasir.

Latihan 2: Pembuatan Logger Otomatis menggunakan Fungsi `init()`

Deskripsi Soal: Modul keuangan dan audit membutuhkan pencatatan log transaksi e-commerce secara terpusat. Buatlah sistem logger yang langsung mengumumkan kesiapannya saat sistem backend dinyalakan.

Kriteria Implementasi:

1. Buatlah direktori baru `/internal/logger` dan buat berkas `logger.go` di dalamnya.
2. Di dalam berkas `logger.go`, deklarasikan fungsi `init()` yang otomatis mencetak pesan konfigurasi sistem audit: `"[AUDIT SYSTEM] Menginisialisasi modul pencatatan transaksi terpusat..."`.
3. Buat fungsi `LogTransaction(orderID string, amount float64)` (Exported) yang mencetak informasi detail transaksi secara formal: `"[AUDIT SYSTEM - %s] Transaksi berhasil tercatat. Nilai: Rp %.2f"`, di mana `%s` diisi oleh tanggal/jam transaksi saat itu (gunakan package bawaan `time.Now().Format("2006-01-02 15:04:05")`).
4. Import package `go-shop/internal/logger` di dalam berkas `main cmd/api/main.go`. Panggil fungsi `LogTransaction` setelah seluruh simulasi checkout belanja di `main()` selesai dijalankan.
5. Perhatikan baris keluaran terminal saat program dijalankan. Buktikan bahwa pesan inisialisasi di dalam `init()` tercetak terlebih dahulu di bagian atas sebelum teks pembuka `main()` dieksekusi.